Server-Side Functions

James Gallagher
OPeNDAP

Software Developer's Workshop for the Australia Bureau of Meteorology (BOM)
October 17, 2007
Melbourne, Australia

Background on DAP2 and Constraint Expressions

- DAP2 interface is via the URL
- The URL includes the
  - Name of the data set (/data/nc/fnoc1.nc)
  - Name of the desired response (DAS, …)
  - Optional Constraint Expression
- Constraint expression chooses which parts of the dataset (granule) are returned
  - If the dataset has several variables, chooses among those
  - If the variables are arrays, subsets those
  - If the are Structures, chooses the fields
  - If there are Sequences, selects values
  - Also runs functions if they are available

Constraint Expression (CE) Syntax

- URL: http://host/dataset.<ext>?<ce>
- If <ext> is ‘dds’ or ‘dods’ the <ce> is used to contrain the response
- <ce> is a <projection>[&<selection>]*
- <projection>
  - is a comma separated list of variables to be returned.
  - can also be an array hyperslab
  - Structure field

CE Projection Examples

- Using the http://localhost:8080/data/nc/fnoc1.nc dataset:
  - ?u,v Chooses the variables u and v
  - ?u[0][0:5][0:5] Chooses u and subsamples the array returning a 6 by 6 hyperslab
- Try these in a browser - you do not need to use the virtual machine's server; you can substitute test.opendap.org for localhost
  - e.g.,http://test.opendap.org/opendap/data/nc/fnoc1.nc.ascii?u[0][0:5][0:5]
  - I used '.ascii' instead of '.dods' so the result is easy to see in a browser

More CEs

- http://test.opendap.org:8080/opendap
- /data/nc/coads_climatology.nc.dds?SST

```
Dataset {
    Grid {
      Array:
        Float32 SST[TIME = 12][COADSY = 90][COADSX = 180
      Maps:
        Float64 TIME[TIME = 12];
        Float64 COADSY[COADSY = 90];
        Float64 COADSX[COADSX = 180];
    } SST;
} coads_climatology.nc;
```

Choosing fields

- http://test.opendap.org:8080/opendap
- /data/nc/coads_climatology.nc.dds?SST.TIME

```
Dataset {
    Float64 TIME[TIME = 12];
} coads_climatology.nc;
```

CEs for data

- Replace the '.dds' in the previous URLs with .ascii or .dods (use ascii with a web browser or dods with an application that understands binary data
- Look at the results
- Try various combinations

What About Selections?

- Selections are used to choose values from relational types
- Try it:
  - http://localhost:8080/opendap/data/ff/avhrr.dat.asc?day
  - http://localhost:8080/opendap/data/ff/avhrr.dat .asc?day&day=19
  - http://localhost:8080/opendap/data/ff/avhrr.dat .asc?&day=19
- The first CE is a projection that chooses just the variable ‘day’ while the second CE includes a selection clause which chooses only those entries where the value of day is 19. The third CE returns all the fields but only those rows where day is 19

But there's more to Constraint Expressions

- They can invoke functions
- The functions can compute and return values
- They can also return a Boolean value for use in the selection part of the constraint expression

CE Functions

- Functions are bound to the server
- Each handler can load it’s own set of functions
- The libdap library (which is used by all of our handlers) includes a base set of functions
- The functions are bundled with the library - we would like to make them run-time loaded modules

CE Function syntax

- <ce> can be a function call
  - <function> “(“ <zero or more args> “)”
- Example: http://… .ascii?version()
  - Run this example in a browser or command shell using getdap
- The version() function is placeholder for a ‘discovery’ mechanism - what would you do to improve it?

```
otaku:~ jimg$ getdap "http://test.opendap.org/opendap/data/nc/coads_climatology.
nc.ascii?version()"
Dataset: virtual
version, "<?xml version=\"1.0\" encoding=\"UTF-8\"?>                          <func
```

More Useful CE Functions

- grid(): sample a Grid variable based on the values of its map vectors
- geogrid(): just like grid(), but assume that the Grid holds geospatial data
- geoarray(): like geogrid() but for array data - some geospatial extent information must be provided
- linear_scale(): Scale data using y=mx+b

Example

- Use the geogrid() function to select values from the COAD Climatology data set:
  - ?geogrid(SST,-5,-80,-40,-50)

```
Dataset: virtual
SST.COADSX, -81, -79, -77, -75, -73, -71, -69, -67, -65, -
SST.SST[SST.TIME=366][SST.COADSY=-41], 13.8763, 13.8154, 1
SST.SST[SST.TIME=366][SST.COADSY=-39], 14.6326, 14.8623, 1
SST.SST[SST.TIME=366][SST.COADSY=-37], 16.4309, 16.6732, 1
SST.SST[SST.TIME=366][SST.COADSY=-35], 18.1081, 18.3181, 1
SST.SST[SST.TIME=366][SST.COADSY=-33], 19.81, 19.6346, 19.
SST.SST[SST.TIME=366][SST.COADSY=-31], 20.7219, 20.6991, 2
SST.SST[SST.TIME=366][SST.COADSY=-29], 21.6793, 21.2337, 2
SST.SST[SST.TIME=366][SST.COADSY=-27], 22.3954, 22.1973, 2
SST.SST[SST.TIME=366][SST.COADSY=-25], 22.9888, 22.8466, 2
SST.SST[SST.TIME=366][SST.COADSY=-23], 23.6531, 23.6586, 2
SST.SST[SST.TIME=366][SST.COADSY=-21], 24.3893, 24.328, 24
SST.SST[SST.TIME=366][SST.COADSY=-19], 25.1083, 25.1151, 2
SST.SST[SST.TIME=366][SST.COADSY=-17], 26.0377, 26.0172, 2
SST.SST[SST.TIME=366][SST.COADSY=-15], 26.8277, 27.0424, 2
SST.SST[SST.TIME=366][SST.COADSY=-13], 27.4275, 27.6395, 2
SST.SST[SST.TIME=366][SST.COADSY=-11], 28.0776, 28.3415, 2
SST.SST[SST.TIME=366][SST.COADSY=-9], 28.2153, 28.067, 28.
SST.SST[SST.TIME=366][SST.COADSY=-7], 28.2379, 28.2117, 28
SST.SST[SST.TIME=366][SST.COADSY=-5], 28.6721, 28.5094, 28
SST.SST[SST.TIME=1096.485][SST.COADSY=-41], 14.2203, 14.57
SST.SST[SST.TIME=1096.485][SST.COADSY=-39], 15.5356, 15.44
SST.SST[SST.TIME=1096.485][SST.COADSY=-37], 17.5115, 17.31
SST.SST[SST.TIME=1096.485][SST.COADSY=-35], 19.1685, 18.83
SST.SST[SST.TIME=1096.485][SST.COADSY=-33], 19.8907, 19.90
```

Observations

- Note that the lat/lon values from the call match those of the response
- The arguments are a little odd but help is available by calling the function with no arguments
- This function is pretty crude, but the same interface can be used in front more sophisticated processing (i.e., GIS)
- Calling the function with the single argument “version” should return the version information for the function (but that seems broken and is probably a poor design choice)
- Note also that this made no choice about the time information in the SST variable…

Selecting TIME

- ?geogrid(SST,40,-80,5,-50,\”2000<TIME\”)
- Try it - be careful with the quotes
- The general rule for geogrid() is that the first five arguments are required and then subsequent arguments are relational expressions which include constants and variables which are the names of the Grid’s map vectors.
- The result is the intersection of the zero or more relational expressions

How to Write CE Functions: short version

- Write a C++ function which uses one of the three CE function type signatures
- Add the function to the constraint evaluator class
- Compile/link the new code into a handler

A Simple Example

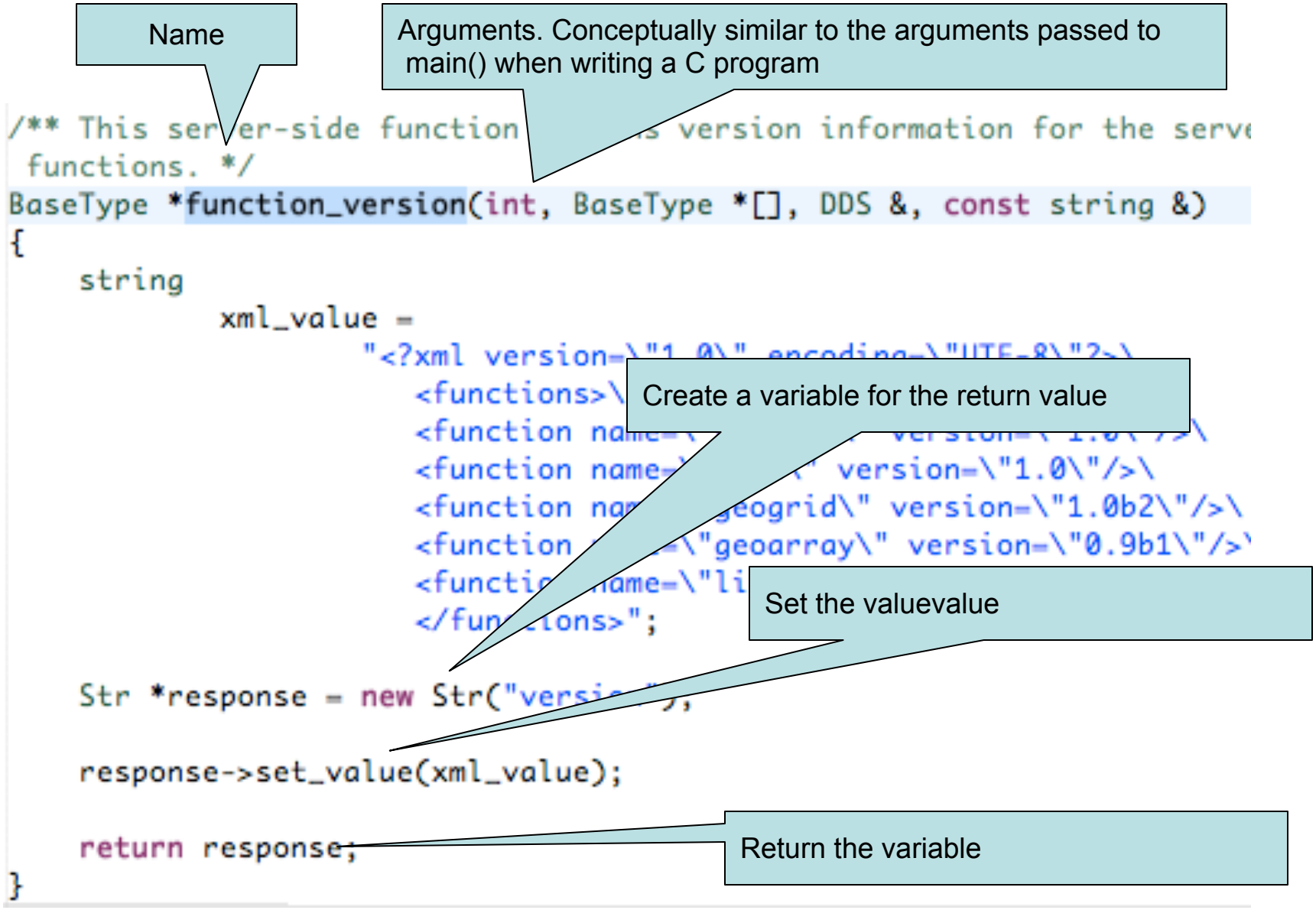
- Look at the libdap source file ce_functions.cc and scan the function “function_version”

```
/** This server-side function returns version information for the serve
 functions. */
BaseType *function_version(int, BaseType *[], DDS &, const string &)
{
    string
            xml_value =
                    "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\
                       <functions>\
                       <function name=\"version\" version=\"1.0\"/>\
                       <function name=\"grid\" version=\"1.0\"/>\
                       <function name=\"geogrid\" version=\"1.0b2\"/>\
                       <function name=\"geoarray\" version=\"0.9b1\"/>\
                       <function name=\"linear_scale\" version=\"1.0b1\
                       </functions>";

    Str *response = new Str("version");

    response->set_value(xml_value);

    return response;
}
```


Name
Arguments. Conceptually similar to the arguments passed to main() when writing a C program
/** This server-side function
version information for the serve
functions. */
BaseType *function_version(int, BaseType *[], DDS &, const string &)
{
string
xml_value =
"<?xml version=\"1.0\" encoding=\"UTF-8\"?>\
<functions>\
<function name=\"
version=\"1.0\"/>\
<function name=\"
version=\"1.0\"/>\
<function name=\"geogrid\" version=\"1.0b2\"/>\
<function name=\"geoarray\" version=\"0.9b1\"/>\
<function name=\"li
</functions>";
Create a variable for the return value
Str *response = new Str("versio
response->set_value(xml_value);
Set the valuevalue
return response;
Return the variable
}

Inserting the Function into the Constraint Evaluator

- Constraint Expressions are evaluated using the ConstraintEvaluator C++ class
- The ConstraintEvaluator::add_function method is used to add a function and bind that function to a symbolic name used in the constraint expression
  - From the ce_functions.cc source file:
  - `ce.add_function("version", function_version);`

Functions are added to the ConstraintEvaluator

- Look in the ConstraintEvaluator.cc and ce_functions.cc source files to see how the base set of functions are added to the default constraint evaluator

```
ConstraintEvaluator::ConstraintEvaluator()
{
    register_functions(*this);
}
```

Programming with libdap

- The library has classes for the different datatypes (Byte, Int16, …, Structure, Grid)
- Each of those are children of BaseType
  - The common OO pattern of using a generic parent class to pass more specific instances is used often by libdap
- The DDS and DAS responses has corresponding objects in libdap
- The DDS holds all of the variables of the data set
  - In recent versions of libdap, the variables in the DDS contain the attribute information but hold no data until either data values are read or until just before the server runs the send_data() method.

More programming with libdap

- The variables held by a DDS may contain data (but they do not have to)
- Data values are read using BaseType::read() methods which are specialized for each handler
  - The Array::read() method for the netCDF handler is different than the Array::read() method in the HDF4 handler

How CE are Evaluated

- When a CE is sent to a server, it first parses the expression (see ConstraintEvaluator::parse_constraint() and the parser in ce_expr.y
- This parses the projection information and marks the variables in the DDS which are part of the projection
  - The ‘send_p’ flag is used for this purpose
  - Each variable has a send_p flag and the send_p() accessor and set_send_p() mutator can be used to access and alter its value.
  - Thus while the evaluator sets the flag based on the parse of the projection information, other code which has access to the variables can also work with this flag

What a CE Function can Expect

- The DDS will be complete
  - It will contain all of the variables in the dataset
  - There will be no data in the variables
  - Variables to be sent will have ‘send_p’ set
  - Each variables will have its attributes set

Other steps you'll need to take

- Once you've written a CE function…
  - Compile it
  - Write the software to add it to an instance of ConstraintEvaluator
  - Recompile/link the relevant code
  - Restart the server

Where should I put my functions?

- You can bundle them with libdap - as the base set of functions are
- You can add them into a handler
  - The FreeForm handler includes a fairly large set of functions - open that software and look for the mechanism used to load them

Different ways to load CE Functions

- They can be added by the ConstraintEvaluator constructor in libdap or by a handler as is the case with FreeForm

```
bool FFRequestHandler::ff_build_data(BESDataHandlerInterface & dhi)
{
    BufPtr = 0;                 // cache pointer
    BufSiz = 0;                 // Cache size
    BufVal = NULL;              // cache buffer

    BESDataDDSResponse *bdds =
        dynamic_cast <BESDataDDSResponse *>(dhi.response_handler->ge
    DataDDS *dds = bdds->get_dds();
    ConstraintEvaluator & ce = bdds->get_ce();

    try {
        FFTypeFactory *factory = new FFTypeFactory;
        dds->set_factory(factory);

        ff_register_functions(ce);
```

Overwriting Functions

- The ConstraintEvaluator::add_function method maintains the function name/ code binding so that newer entries with the same name take precedence.
- Thus a specific handler can override the definition of one of the base-set functions

A ‘Real’ CE Function

- Look at `function_linear_scale` in `ce_functions.cc`
- Synopsis:
  - The function does some fancy processing with arguments - if arguments are passed it will use those values, if not it will look at the attributes for the scale factor (m) and add offset (b)
  - The scaling operation is slightly different for Grids and Arrays
  - Because the variable name was passed in via a function call list, the expression parser did not set the send_p flag - the function sets it
  - Then the function reads the data values from the data set into the variable

Argv[0] is first argument to function and is the variable to scale. Here we use the BaseType method Is_vector_type() to determine that argv[0] is an Array and then use a C++ trick to cast that BaseType pointer down the class hierarchy to an Array. This makes it easier to call methods specific to the Array class

```
} else if (argv[0]->is_vector_type()) {
    Array &source = dynamic_cast<Array&>(*argv[0]);
    source.set_send_p(true);
    // If the array is really a map, make sure to read using the Grid
    // because of the HDF4 handler's odd behavior WRT dimensions.
    if (source.get_parent() && source.get_parent()->type() == dods_grid_c)
        source.get_parent()->read(dataset);
    else
        source.read(dataset);
```

As the comment says, if the Array is really a Grid map variable, be sure to use Grid::read() to load data values into variable. If not, just call Array::read()

Extract_double_array() is a helper method that reads numerical values from a DAP variable and loads them into a dynamically allocated double array. It's defined in the ce_functions.cc source file

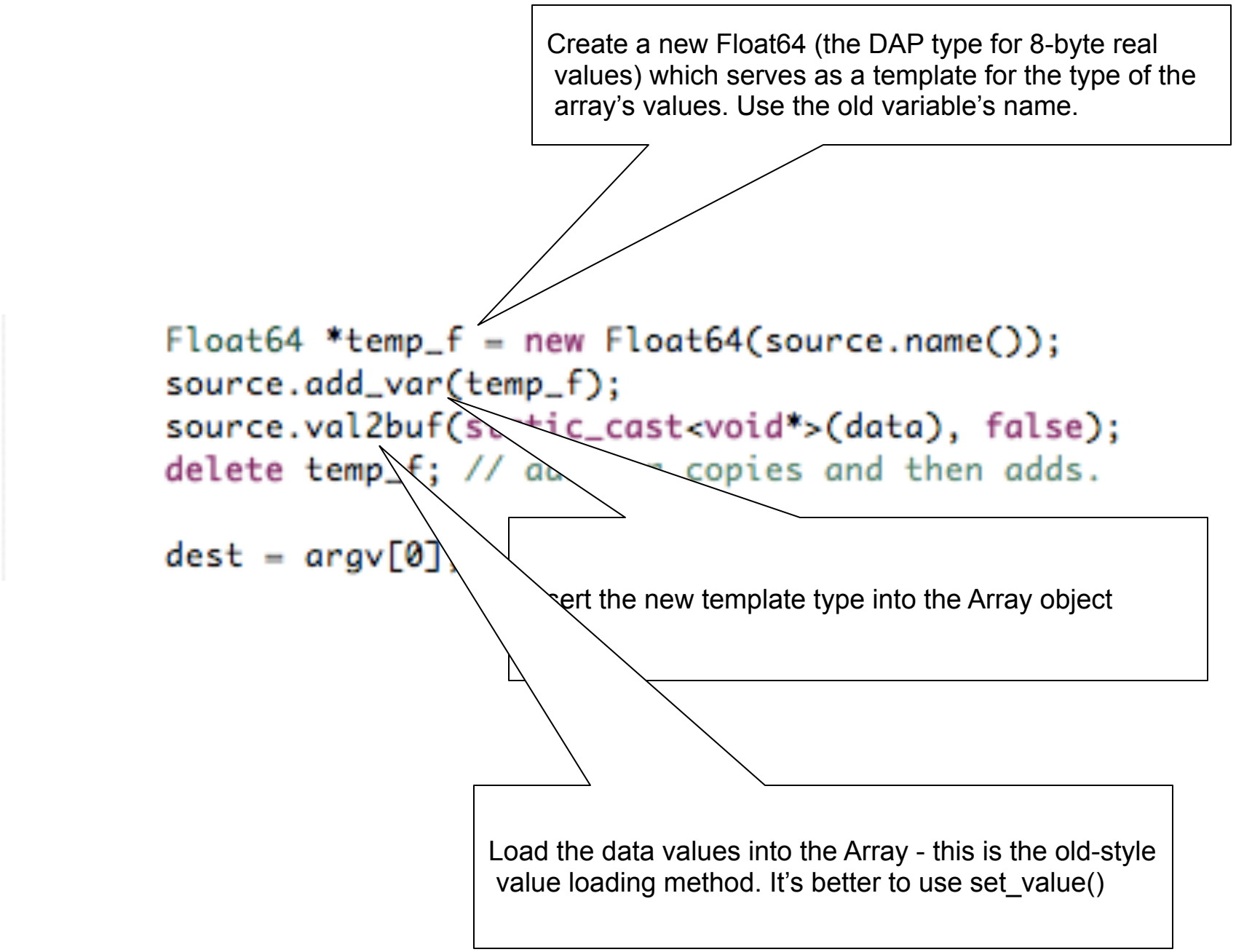
The Array::length() methods returns the number of elements in an array

```
data = extract_double_arr...(&source);
int length = source.length();
int i = 0;
while (i < length) {
    if (!use_missing || !double_eq(data[i], missing))
        data[i] = data[i] * m + b;
    ++i;
}
```

Y=mx+b

More…

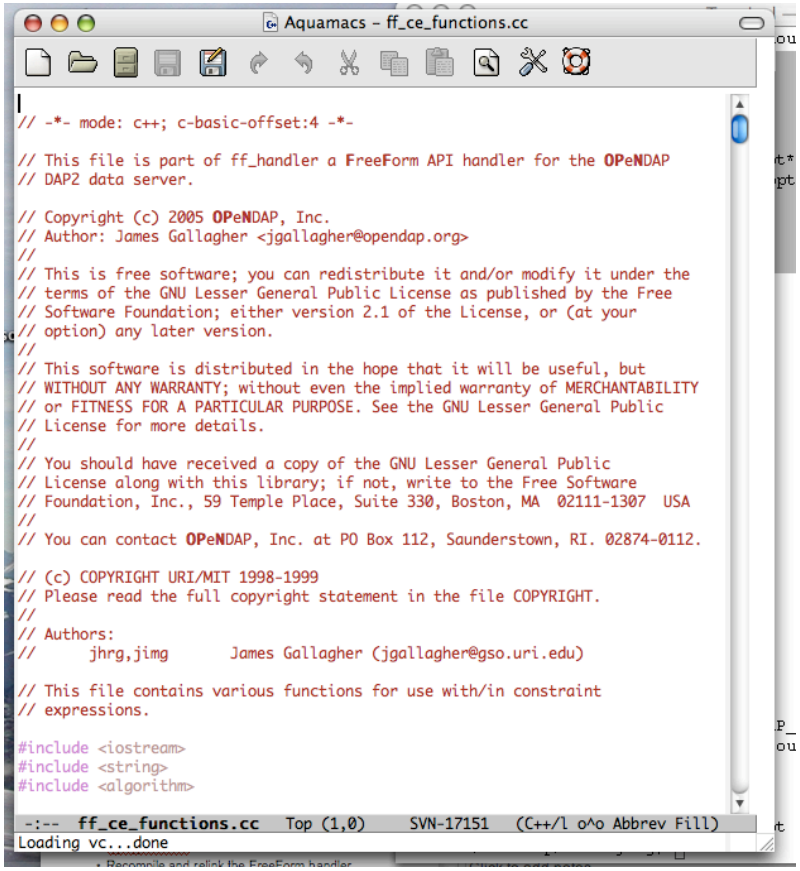
- Then the values are copied from the variable in the DDS to an Array of doubles
  - This array will hold the result of the scaling operation
  - The values are scaled
  - A new DAP array of Float64 is created and the scaled values are stored there
  - The newly created DAP Float64 array is returned


Create a new Float64 (the DAP type for 8-byte real values) which serves as a template for the type of the array's values. Use the old variable's name.
Float64 *temp_f = new Float64(source.name());
source.add_var(temp_f);
source.val2buf(static_cast<void*>(data), false);
delete temp_f; // ad copies and then adds.
dest = argv[0];
sert the new template type into the Array object
Load the data values into the Array - this is the old-style value loading method. It's better to use set_value()

Write a HelloWorld Function

- Write the function in the FreeForm CE functions file ff_ce_functions.cc
- Recompile and relink the FreeForm handler
- Install the modified handler
- Test

Open `ff_ce_functions.cc`



Add the function

- At this step it's just a stub
- The code should still compile - check it by typing 'make'

```
}

BaseType*
function_hw(int argc, BaseType *argv[], DDS &dds, ConstraintEvaluator &ce)
{
}

void ff_register_functions(ConstraintEvaluator & ce)
```

Now add the function's guts

- Use C++'s string class to hold the value
- Create a new DAP 'Str' (String) variable
- Load the value
- Return the Str instance - Str is a child of BaseType

```
BaseType*
function_hw(int argc, BaseType *argv[], DDS
{
    string stuff = "Hello World!";

    Str *response = new Str("version");

    response->set_value(stuff);

    return response;
}
```

Compile to catch any errors…

Now arrange to register the functions

- Use ConstraintEvaluator::add_function()
- If you add this call to ff_register_functions() then you're all set
- How else could you add the function?

```
void ff_register_functions(ConstraintEvaluator & ce)
{
    ce.add_function("helloworld", function_hw);
```

Compile, Install and …

- Compile: ‘make’
- Install: ‘make install’
  - This will copy the freeform handler shared object library (aka ‘module’) to the directory where the BES expects to find it.

…Test

- Test
  - Start the BES and use the BES command line tool to test the new function
  - bescmdln -p 10002 -h localhost
  - set container in catalog values f,/data/ff/avhrr.dat;
  - define d1 as f;
  - get ascii for d1;
    - Lots of output…
  - define d2 as f with f.constraint=“helloworld()”;
  - get ascii for d2;
    - Dataset: virtual
    - Hw, “Hello World!”

Now Start Tomcat and Complete the Tests

- Start up Tomcat - now you can the with the web server
- Try it in a web browser
  - http://localhost:8080/opendap/data/ff/avhrr.dat.ascii?helloworld()
  - In a browse, append '.ascii' by hand
- Try the same URL with getdap in a command line tool
  - getdap -D http://…/avhrr.dat -c "helloworld()"
  - With getdap, use -D (data) and -c (constraint)
  - Make sure to quote the CE since parens are a shell meta-character

Summary

- A Server-side function is a function added to the constraint evaluator
- The function has access to all of the variables in the data set
- The return value of the function is
  - Encapsulated in a DAP variable
  - Or a Boolean value
  - Or void
- We wrote a simple example of the first type